Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C_1 We characterize the latency, bandwidth, and jitter sensitivity of AI communication workloads at scales up to 4,096 GPUs.
- C_2 We introduce a compositional linear-programming method with signature-based caching for practical large-scale analysis.
- C_3 We use the results to derive network performance envelopes and identify useful provisioning points.

B. Computational Artifacts

The evaluation uses two artifacts:

- A_1 Code, compact data, scripts, and results: https://github.com/tommasobo/SC_Tracing, branch artifact_freeze.
- A_2 Public input traces: http://storage2.spcl.ethz.ch/traces/ai/.

Artifact	Contributions	Paper elements
A_1	C_1, C_2, C_3	Figs. 1, 3–10
A_2	C_1, C_2	Table II, Figs. 3–6

The requested badges are Artifacts Available, Artifacts Evaluated-Functional, and Results Reproduced. A persistent DOI will be added at the artifact-freeze stage.

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

Artifact A_1 contains the solver, trace-processing tools, compact plotting data, reproduction scripts, verification manifests, and final plots. The quick workflow redraws Figures 1 and 3–10. Figure 2 is a method diagram. The full workflow also runs bounded checks for Figures 3, 4, and 6 and verifies the closest recovered Figure 5 result.

Expected Results

The quick workflow should produce eight PDF files under figures/. The full workflow should also produce task manifests and result CSV files in the selected scratch directory.

Figures 1, 7, and 10 are exact redraws of compact data. Figures 8 and 9 redraw the published values embedded in their plotting script. The paper-style plots for Figures 3, 4, and 6 use the compact paper data, while separate raw-derived checks record the reproducibility of those experiments. The closest Figure 5 Composite curve differs from the paper CSV by at most 0.014853%.

The following limitations are documented in docs/REPRODUCTION_REPORT.md:

- the exact paper-era Figure 5 per-motif transformation was not recovered;
- the available Figure 6 Llama metadata identify Llama 7B at 128 GPUs, while the paper says Llama 70B;
- the Figure 6 vLLM full curves require an unavailable paper-era token-window reduction; and
- Grok at 4,096 GPUs is plot-only unless the expensive option is explicitly enabled.

Expected Reproduction Time (in Minutes)

Installation takes about 10 minutes. The quick workflow takes under 5 minutes and took about 20 seconds on Alps. The default full workflow takes under 6 hours with independent tasks submitted in parallel. Each individual default experiment is bounded to about 45 minutes. Grok at 4,096 GPUs is excluded from this estimate.

Artifact Setup (incl. Inputs)

Hardware: The quick workflow needs one CPU core, 2 GB of RAM, and about 100 MB of free space. The full workflow was tested on Alps CPU nodes. We recommend 4 or more CPU cores, 32 GB of RAM, and a scratch filesystem. No GPU is required. The Grok 4,096-GPU analysis needs substantially more memory and time and is disabled by default.

Software: Python 3.10 or newer is required (https://www.python.org/). Package versions are listed in requirements.txt; full verification also uses requirements-dev.txt. Fresh solver runs need Gurobi and a license (https://www.gurobi.com/). LogGOPSim is included in the repository and additionally uses g++, gengetopt, and re2c. Slurm is optional.

Datasets / Inputs: Compact data are committed under data/, local_artifact/results/, and results/reproduced/. Hashes and raw-input identities are under local_artifact/manifests/. Large NSYS, SQLite, GOAL, solver cache, and temporary files are intentionally kept outside Git.

Installation and Deployment:

```
git clone https://github.com/tommasobo/SC_Tracing.git
cd SC_Tracing
git checkout artifact_freeze
python3.11 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
```

For the full validation, also install requirements-dev.txt and confirm that Gurobi can obtain a license.

Artifact Execution

The default local check is:

```
./reproduce_quick.sh
```

Run the full workflow from a scratch filesystem:

```
./reproduce_full.sh \
  --scratch /scratch/$USER/provisioning_artifact \
  --workers 4
```

On Alps, submit independent tasks through Slurm:

```
./reproduce_full.sh --slurm \
  --account <account> --partition normal \
  --scratch /iopsstor/scratch/cscs/$USER/artifact \
  --workers 4
```

The full launcher serializes the longest solver sweeps when needed to respect license limits. Grok at 4,096 GPUs is run only with –expensive_run and an explicit N1024 analysis directory.

Artifact Analysis (incl. Outputs)

The quick workflow writes the final PDFs under figures/ and checks their presence. The full workflow writes one manifest per task, fresh CSV files, logs, and a summary in the selected scratch directory. The final comparison and known differences are in docs/REPRODUCTION_REPORT.md.

B. Computational Artifact A_2

Relation To Contributions

Artifact A_2 contains the execution traces used to build communication graphs and validate the analysis pipeline. It supports the trace-based checks for Figures 3–6.

Expected Results

A selected trace should download with the recorded hash and pass through SQLite export, GOAL conversion, binary conversion, LogGOPSim replay, and plotting. The repository includes provenance for the exact inputs used in the final checks.

Expected Reproduction Time (in Minutes)

A small trace download and replay takes about 5–45 minutes. Processing a large campaign can take several hours and is not required for the functional check.

Artifact Setup (incl. Inputs)

Hardware: Use one CPU node with at least 32 GB of RAM and a scratch filesystem.

Software: The trace workflow uses curl, SQLite, the repository's Python tools, and LogGOPSim. NSYS is needed only when exporting a new report.

Datasets / Inputs: The traces are listed at http://storage2.spcl.ethz.ch/traces/ai/. Exact filenames and SHA-256 hashes used by this artifact are recorded under local_artifact/manifests/.

Installation and Deployment: Install Artifact A_1, choose one trace recorded in the manifests, and download it to scratch. Do not store large trace files in the repository or the home directory.

Artifact Execution

Follow the commands in docs/provenance/ for the selected trace. The stages are NSYS or SQLite input, GOAL, binary schedule, LogGOPSim replay, and CSV plotting. A moderate trace is sufficient for evaluation.

Artifact Analysis (incl. Outputs)

Record the downloaded file's SHA-256 hash, the replay runtime, and the generated CSV. These values can be compared with the committed manifest and reference result.

Artifact Evaluation (AE)

A. Computational Artifact A_1

Artifact Setup (incl. Inputs)

Clone branch artifact_freeze, create the Python environment, and install requirements.txt. For full validation, install requirements-dev.txt, verify the Gurobi license, and choose a scratch directory. On a cluster, use a compute allocation or the Slurm launcher rather than a login node.

Artifact Execution

First run:

```
./reproduce_quick.sh
```

It redraws Figures 1 and 3–10 from the committed compact inputs and validates the data manifests. Then run either the local full command or the Slurm command from the AD section. Do not add –expensive_run for the normal evaluation.

The main checks are:

- Figures 3 and 4: compare the fresh bounded runs with the selected raw-derived results and the paper CSVs;
- Figure 5: verify the selected 201-point Composite CSV and its maximum relative error of 0.014853%;
- Figure 6 Llama: run the bounded latency and bandwidth checks with four physical NIC queues; and
- Figure 6 vLLM and Grok 4k: inspect the committed replay evidence and compact data. These are not rerun by default.

Artifact Analysis (incl. Outputs)

Confirm that the eight expected PDF files exist under figures/. For a full run, inspect summary.json, the task manifests, and the result CSVs in scratch. Use docs/REPRODUCTION_REPORT.md for the expected error values and the interpretation of remaining differences.

B. Computational Artifact A_2

Artifact Setup (incl. Inputs)

Install Artifact A_1 and download one trace listed in the committed manifests to scratch. Verify its SHA-256 hash before processing it. The Llama-3.1-8B vLLM trace is the most complete end-to-end example.

Artifact Execution

Use the documented trace pipeline to produce SQLite, GOAL, binary, and LogGOPSim outputs. A single archived vLLM job point is enough to check the pipeline. The available online 70B trace is not a substitute for the paper-era vLLM input.

Artifact Analysis (incl. Outputs)

Compare the input hash and replay runtime with the committed manifest. The archived vLLM point should replay exactly. The full sensitivity curve is expected to differ because the paper-era token-window reduction is unavailable.